

UNIVAI — Requirements (FlowOps-style)

The Autonomous Learning Simulator

Date: February 2026

Audience: Product, Engineering, and Delivery Teams

Document Purpose

This document defines the product and system requirements for UnivAI, an AI-powered learning simulator that converts static textbooks into structured academic semesters with lectures, schedules, attendance enforcement, and automated assessments.

Primary Outcomes

- Transform any uploaded textbook into a semester-based curriculum with weekly lectures.
- Ground all learning content using Retrieval-Augmented Generation (RAG) with citations.
- Enforce attendance and academic discipline using schedule-based penalties and quizzes.
- Provide multimodal learning (audio lectures, diagram explanations, and interactive tutoring).

What the System Is

UnivAI is a structured academic simulator combining an LLM backend, RAG knowledge system, multi-agent orchestration, and multimodal learning delivery. The system ingests a textbook, creates a syllabus, delivers lectures, generates exams, and tracks learner progress.

MVP Scope (Core Modules)

Module 1 — Curriculum Planner (Dean Agent)

- Analyze Table of Contents and generate structured semester schedule.
- Assign lectures, labs, and exams across weeks.

Module 2 — Lecturer Copilot (RAG)

- Retrieve textbook passages using hybrid search (keyword + vector).
- Generate lectures and explanations grounded in cited pages.

Module 3 — Classroom Delivery (Multimodal)

- Convert lecture scripts to audio using Text-to-Speech.
- Explain diagrams and charts using vision models.
- Provide a Teaching Assistant chatbot for live lecture questions.

Module 4 — Assessment System

- Generate quizzes and exams aligned with weekly curriculum.
- Automatically grade answers and provide feedback.

System Architecture (High-Level)

Layered architecture: Web UI → API Gateway → AI Services (Agents, RAG, Lecture Engine)
→ Tools/Integrations → LLM Providers and Data Stores.

- Frontend: Next.js LMS-style dashboard.
- Backend: FastAPI/Django for orchestration.
- Data Stores: PostgreSQL + pgvector embeddings.
- Infrastructure: Dockerized deployment on cloud providers.

Agent Design

Planner (Dean Agent): Generates the academic schedule.

Lecturer Agent: Retrieves content and produces lecture scripts.

Proctor Agent: Creates exams and quizzes.

Grader Agent: Evaluates responses and updates student progress.

TA Agent: Provides real-time tutoring assistance.

Security & Safety

- File validation and input sanitization for uploads.
- Prompt-injection detection and content moderation.

- Rate limiting to prevent abuse and control token costs.
- Hallucination control: responses must cite book sources or state unavailability.

Development Roadmap

Week 1: Ingestion pipeline and vector database setup.

Week 2: Curriculum planner and RAG lecture generation.

Week 3: Multi-agent orchestration and TTS integration.

Week 4: UI polish, testing, deployment, and observability.

Success Metrics

- Faithfulness: >85% citation grounding.
- Latency: Chat responses under 3 seconds.
- Lecture generation time under 1 minute.
- Student completion rate and engagement tracking.